

Oct 7, 2026 · @Girish

Work top to bottom. **Do not start a stage until the gate at the end of the previous stage passes.** If you change anything in an earlier stage, redo every gate after it.

Code files: only `Tuning.java` is staged (staged `Tuning.java`, version 2, 7 Oct 2026). Each **STAGE** part of it matches a stage here. `Constants.java` stays our normal file: at each stage you paste the new AutoTune output over the matching block.

- **Switch on the Foresight tuner (Stage 3):** in `Tuning.java`, delete the `===== STAGE 3 START` line and the `===== STAGE 3 END` line.
- **Switch `tests()` to the next stage:** put `//` in front of the active `return` line, and remove `//` from the next stage's `return` line. Exactly one `return` line is active at a time.
- **Paste AutoTune output into `Constants.java`:** select the whole old block (from `public static ...` to its closing `);`) and paste the new block over it.
- After every code change: build and send to the robot.

Stage 0 — Before you start

- ☐ Battery fully charged. Low battery gives poor tuner values ([Pedro docs: Troubleshooting](#)).
- ☐ Robot on flat field tiles with plenty of room. The braking tests need clear space ahead, behind and to the sides ([ForesightTuner.java](#)).
- ☐ Pinpoint plugged into an I2C port **other than port 0** ([Pedro docs: Pinpoint](#)).
- ☐ Pinpoint mounted **sticker side up** (same page).
- ☐ One pod rolls forward (X pod), one rolls sideways (Y pod) ([goBILDA guide, p. 5](#)).
- ☐ Forward pod in the X port, strafe pod in the Y port (Pedro docs: Pinpoint).
- ☐ Motor and Pinpoint names in the Driver Hub configuration written down.
- ☐ Laptop connected to the Robot Controller; AutoTune opens at `http://192.168.43.1:10158` ([Pedro docs: Tuning](#)).

Code setup (once):

- ☐ Saved a backup of the current `Constants.java` and `Tuning.java` outside the project (for example as `.txt` files in a backup folder).
- ☐ Copied the staged `Tuning.java` into `TeamCode/src/main/java/org/firstinspires/ftc/teamcode/pedro/`, replacing the old one.

- ☐ `Constants.java` has these three blocks with exactly these names: `drivetrainConfig`, `localizerConfig`, `foresightConfig`. (Our current file does.) The staged `Tuning.java` uses all three.
- ☐ The `procedures` folder from the Pedro Quickstart (`MecanumTuner`, `PinpointTuner`, `ForesightTuner`, `Tests`) is in the same `pedro` folder ([Pedro docs: Installation](#)).
- ☐ Built and sent with no errors.

GATE 0: all boxes ticked.

Stage 1 — Drivetrain (Mecanum Procedure)

- ☐ In AutoTune, chose **Mecanum Procedure**. (`mecanumTuner()` is already on in the staged `Tuning.java`.) ([Pedro docs: Mecanum](#))
- ☐ Entered each motor's name in its matching slot (front left, back left, front right, back right).
- ☐ For each of the 4 motors: watched it spin and answered honestly whether it went the intended way.
- ☐ **Constants.java**: pasted the `MecanumConfig` from the **Java** tab over the `drivetrainConfig` block. (Keeping our `HardwareNames` lines is fine if the names and directions match.)
- ☐ **Tuning.java**, `tests()`: the STAGE 1 `return` line (drivetrain only) is the active one. Built and sent.

GATE 1 — Tests → Driving (same page; [Troubleshooting](#)):

- ☐ Left stick Y drives straight forward and back.
- ☐ Left stick X strafes straight left and right.
- ☐ Right stick turns in place.
- ☐ Noted which side leads when driving forward and taped **FRONT** on it. Every later step uses this forward.

Stage 2 — Pinpoint (Pinpoint Procedure)

This is the most important stage. Every later number is measured through the Pinpoint (see *Start here* in the main guide).

- ☐ In AutoTune, chose **Pinpoint Procedure**. (`pinpointTuner()` is already on in the staged `Tuning.java`.) ([Pedro docs: Pinpoint](#))
- ☐ Setup: entered the HardwareMap name and picked the pod type (goBILDA 4-bar for us).

- ☐ **Forward direction:** pushed the robot straight forward, toward FRONT, at least a foot, then stopped the OpMode.
- ☐ **Strafe direction:** pushed the robot straight to **its left** (stand behind it facing FRONT), at least a foot, then stopped the OpMode.
- ☐ **Offsets:** taped a cross on the floor under the robot's centre. Turned the robot slowly **180° counterclockwise**, keeping its centre over the cross, then stopped the OpMode.
- ☐ **Constants.java** : pasted AutoTune's `PinpointConfig` over the `localizerConfig` block.
- ☐ **Tuning.java**, `tests()` : put `//` in front of the STAGE 1 `return` line and removed `//` from the STAGE 2 `return` line (drivetrain + Pinpoint, no algorithm) ([Pedro docs: Localization](#)). Built and sent.

GATE 2a — Directions (Tests → Localization) (Pedro docs: Localization):

- ☐ Position starts at (0, 0).
- ☐ Pushing toward FRONT makes **x go up**.
- ☐ Pushing to the robot's left makes **y go up**.
- ☐ Turning counterclockwise makes the heading go up ([Pedro docs: Coordinates](#)).

GATE 2b — Offsets match the robot ([goBILDA guide, p. 2](#)):

- ☐ Measured with a ruler: X pod sideways distance from centre (left +, right -) = _____ in. File says _____ in.
- ☐ Measured with a ruler: Y pod forward distance from centre (front +, behind -) = _____ in. File says _____ in.
- ☐ Signs match. Values agree within about 1 in (our rule of thumb). If not, re-run the offsets step or use the ruler values.

GATE 2c — Accuracy (Tests → Localization, push by hand):

- ☐ Push 48 in along a tape line: $x \approx 48$, $y \approx 0$. (Our check.)
- ☐ Push back to the start: within about 0.5 in of (0, 0) ([goBILDA guide, p. 7](#)).
- ☐ **Spin test:** spin several times in place, set it back on its mark: x and y return close to 0. If not, the offsets are wrong (see the main guide, *How the tuner finds offsets*).

Stage 3 — Foresight (Foresight Procedure)

Only start when Gates 2a, 2b and 2c all pass. The steps run in this order ([Pedro docs: Foresight](#)).

- ☐ **Tuning.java** : deleted the `STAGE 3 START` and `STAGE 3 END` lines to switch on `foresightTuner()` (it uses our Pinpoint and Mecanum settings). Built and sent.

- ☐ Battery freshly charged.
- ☐ Robot placed with lots of room ahead and to its left. The speed tests drive 48 in at full power by default, then drift on ([ForesightTuner.java](#)). **Our numbers:** at our top speeds, coasting adds roughly 48 in forward and 15 in sideways ($\text{speed}^2 \div (2 \times \text{natural deceleration})$, using our values).

#	Step	What the robot does	Done
1	Forward Velocity	Full power forward 48 in, then drifts past	
2	Strafe Velocity	Full power left 48 in, then drifts past	
3	Forward Deceleration	Forward until 30 in/s, then coasts; short distance	
4	Strafe Deceleration	Left until 30 in/s, then coasts; short distance	
5	Heading Braking	Spins back and forth in place	
6	Heading Tuner	Spins one way for a couple of seconds	
7	Forward Braking	Drives forward 36 in (minimum 15), brakes, returns; repeats at several powers	
8	Strafe Braking	Same, sideways	
9	Forward Translational	Drives forward about 12–24 in	
10	Strafe Translational	Drives left about 12–24 in	

- ☐ Ran the whole procedure **2–3 times** ([Pedro docs: Troubleshooting](#)). Pasted each run's Java tab output into the [Pedro Tuning Checker](#) as Run 1, 2 and 3.
- ☐ **Constants.java**: pasted the checker's averaged `ForesightConfig` over the `foresightConfig` block. Built and sent.

GATE 3 — Sanity checks on the numbers (from the tuner code, [ForesightTuner.java](#)):

- ☐ Runs gave similar values (no single run far off the others).
- ☐ No value is 0, negative or `NaN`.
- ☐ $\text{brake kV} \approx 0.85 \times \text{coast kV}$.
- ☐ $\text{Large kP} \div \text{small kP} \approx 2.71$, for both forward and strafe.
- ☐ $1 \div \text{coast kV}$ is close to `maxAchievableForwardVelocity` (our check; v1 had 56.3 vs 57.6 in/s).

Shortcut: paste your runs into the [Pedro Tuning Checker](#). It runs these checks, compares runs, checks the offset signs against where the pods really are, and gives the averaged config.

Stage 4 — Wire the constants in

- ☐ **Constants.java**, **create()**: the third argument of `new Follower(...)` is `new Foresight(foresightConfig)`, **not** `null` ([Pedro docs: Constants](#)). Our file still has `null //new Foresight(foresightConfig):delete null //` so the line reads `new Foresight(foresightConfig)`.
- ☐ **Tuning.java**, **tests()**: put `//` in front of the STAGE 2 `return` line and removed `//` from the STAGE 4 `return` line, which passes `() -> new Foresight(Constants.foresightConfig)` ([Pedro docs: Test](#)).
- ☐ Built and sent with no errors.

GATE 4:

- ☐ Tests opens the Hold and Line tests without the message "Algorithm is required..." ([Tests.java](#)).
- ☐ Searched `Constants.java` for `null`: it no longer appears in `create()`.

Stage 5 — Test the follower (Tests Procedure)

Run the tests in this order ([Pedro docs: Test](#)). The docs do not give pass marks, so the pass marks below are ours.

Hold Test — the robot holds its starting spot.

- ☐ Pushed off course by hand, it returns to its spot and heading.
- ☐ It settles without shaking back and forth.

Line Test — forward 48 in and back, again and again, always facing the same way.

- ☐ Set up as in the main guide (*Setting up the robot for a test*).
- ☐ Stays on the tape line for at least 5 laps, with no sideways drift that grows lap after lap.
- ☐ Heading stays the same the whole time.
- ☐ At the far end, x on the screen matches the floor. Carry-past measured: _____ in.

Curved Test — a quarter circle to a point 48 in ahead and 48 in left, facing along the path, then back.

- ☐ Follows the curve smoothly and ends near the same points each lap.

Interpolation Test — the same curve with different heading styles.

- ☐ Turns as each heading style asks while staying on the curve.

If any test fails, go to *Fixing the Line Test* (Step 5 table) and *Our Line Test investigation* in the main guide. If the fix changes Stage 1 or 2, redo every gate from there.

Sign-off log

Gate	Date	Checked by	Notes
Gate 0 — Before you start			
Gate 1 — Driving			
Gate 2a — Directions			
Gate 2b — Offsets			
Gate 2c — Accuracy and spin			
Gate 3 — Foresight numbers			
Gate 4 — Wired in			
Hold Test			
Line Test			
Curved Test			
Interpolation Test			